Lab 3C: CI/CD - Claude in Your Pipeline

Duration: 25-30 min

After: Production & CI/CD lecture (Day 3, Block 3)

Prerequisites: Lab 3B complete; GitHub account + `gh` CLI for Phase 2 (Phase 1 needs no GitHub)

Objective

Run Claude Code headless with `claude -p` under explicit `--max-turns` and `--allowedTools` budgets, make the run **idempotent** (safe to re-run) and **fail-safe** (non-zero exit when the cost bound is hit), parse structured JSON output, then wire the official `anthropics/claude-code-action@v1` GitHub Action for **advisory** PR review.

Deliverable (toolkit chain 11 of 12)

A headless review script (`scripts/ci-review.sh`) with an idempotency guard and hard cost bounds, and a `.github/workflows/claude-review.yml` advisory workflow in the toolkit repo.

> ***START MARKER:*** *Regular terminal (NOT inside Claude),* `cd business-dashboard`*.*

Phase 1: Headless Mode (15 min)

Headless = one prompt in, output out, exit code, no conversation. This is Claude as a tool other tools can call.

Step 1 - basic invocation:

```
claude -p "List all JS files in src/ and describe each in one sentence." --allowedTools
"Read,Glob"
```

Expected output marker: a file list prints, then the process exits - no interactive prompt returns.

> ***Two flags you must never omit in CI:***

- `--allowedTools "Read,Grep,Glob"` - headless has no human to approve tools; without this Claude can't read files (or with looser settings could do too much).
- `--max-turns 10` - caps the agentic loop so a confused run can't burn tokens forever. This is your **cost bound**: a run that hits it must FAIL the job (non-zero exit), never silently retry - Step 4 wires that in.

Step 2 - pipe input:

```
git log --oneline -10 | claude -p "Generate a markdown changelog from this git log.
Sections: Features, Fixes, Chores." --max-turns 3
```

Expected output marker: a formatted changelog on stdout.

Step 3 - structured JSON:

```
claude -p "Review src/routes/tasks.js. Return JSON:
{\"issues\":[{\"severity\":\"high|medium|low\",\"message\":\"...\"}],\"score\":number}" \
--allowedTools "Read,Grep,Glob" --max-turns 10 --output-format json > review.json
cat review.json
```

Expected output marker: a JSON envelope with `type`, `subtype`, `result`, `session_id`, `total_cost_usd`, `duration_ms`, `num_turns`. Your review text is a string inside `.result`:

```
jq -r '.result' review.json
jq -r '.total_cost_usd' review.json
```

Step 4 - the reusable gate script, now with hard bounds. Create `scripts/ci-review.sh`:

```
#!/bin/bash
set -e
MAX_TURNS=10

# --- IDEMPOTENCY GUARD: safe re-run (Step 5 explains) ---
SHA=$(git rev-parse HEAD)
MARKER=".ci-review/${SHA}.done"
if [ -f "$MARKER" ]; then
echo "SKIP: HEAD ${SHA:0:7} already reviewed - re-run is free."; exit 0
fi

claude -p "Review the code in src/ for security and quality issues. Return JSON: {\"issues\
":[{\"severity\":\"critical|high|medium|low\",\"message\":\"...\"}],\"score\":number}" \
--allowedTools "Read,Grep,Glob" \
--max-turns $MAX_TURNS \
--output-format json > review.json

# --- COST BOUND FAIL-SAFE: hitting the turn cap is a FAILURE, not a soft landing ---
SUBTYPE=$(jq -r '.subtype // empty' review.json)
TURNS=$(jq -r '.num_turns // 0' review.json)
if [ "$SUBTYPE" = "error_max_turns" ] || [ "$TURNS" -ge "$MAX_TURNS" ]; then
echo "COST BOUND HIT (${TURNS}/${MAX_TURNS} turns) - failing safe so CI does NOT
auto-retry."
exit 2
fi

SCORE=$(jq -r '.result' review.json | jq -r '.score // 0')
COST=$(jq -r '.total_cost_usd' review.json)
echo "Review score: $SCORE"
echo "Cost: \$$COST"
if (( $(echo "$SCORE < 6" | bc -l) )); then
echo "REVIEW FAILED (score < 6)"; exit 1
fi

# --- success: leave the marker so the next run on this SHA is a no-op ---
mkdir -p .ci-review && touch "$MARKER"
echo "REVIEW PASSED"
```

```
chmod +x scripts/ci-review.sh && ./scripts/ci-review.sh
```

Expected output marker: `Review score: <n>`, a cost line, and `REVIEW PASSED` (exit 0) - or a non-zero exit you can gate on: `1` = quality gate failed, `2` = cost bound hit. This exact script is your capstone CI check tomorrow. It works in ANY CI system: GitHub, GitLab, Azure, Jenkins.

Step 5 - prove the idempotency guard (safe re-run):

```
./scripts/ci-review.sh # second run, same HEAD
echo "exit code: $?"
```

Expected output marker: `SKIP: HEAD <sha> already reviewed - re-run is free.` and exit code `0` - **no tokens spent**. CI re-fires workflows constantly (synchronize events, manual re-runs, flaky runners); the SHA marker means a re-run never bills twice, posts twice, or edits twice. Commit something small and run again to see a fresh review fire on the new SHA. (Add `.ci-review/` to `.gitignore` - the marker is runner-local state.)

> ***Loop-to-invoice alerting:*** *the two guards above stop a single runaway run. The failure mode that makes invoices is the* ***loop****: a bounded run fails, CI auto-retries it, and the same bill repeats every few minutes all weekend. Log* `total_cost_usd` *from every run and alert a human channel when a single run exceeds a ceiling (e.g. $0.50) or a pipeline's daily total crosses a budget - the alert must fire* ***before*** *the invoice, not on it. Minimum viable version: one* `if` *on* `$COST` *that posts to your team channel; never rely on the monthly bill as your alarm.*
>
> ***Billing note:*** *this lab uses Claude Code print mode and a CI pipeline, not the Claude Agent SDK. Confirm the runner credential: subscription CLI usage follows the live account allowance; API/cloud credentials are token billed. Eligible Agent SDK plan users may have a separate* ***monthly*** *SDK credit, but that is a different execution path.*

Phase 2: Pipeline Review - Advisory (10-15 min)

Your shop runs Azure DevOps, so that is the primary path here. The GitHub version follows it as an alternate - the mechanics are the same in both, and only the plumbing differs. Phase 1 was the core learning either way: everything below just moves that same headless call onto a server.

Azure DevOps Pipelines (primary)

Step 1 - store the credential. In your project: **Pipelines -> Library -> + Variable group**, name it `claude`, add `ANTHROPIC_API_KEY`, and click the padlock to mark it **secret**. Link the group to your pipeline.

Step 2 - the pipeline. Create `azure-pipelines.yml` at the repo root:

```
trigger: none
pr:
branches:
include: [ main, master ]

pool:
vmImage: ubuntu-latest

variables:
- group: claude

steps:
- checkout: self
fetchDepth: 0

- script: curl -fsSL https://claude.ai/install.sh | bash
displayName: Install Claude Code

- script: |
git diff --name-only origin/$(System.PullRequest.TargetBranch)...HEAD > /tmp/changed.txt
claude -p "Review the files listed in /tmp/changed.txt for security, quality, and test
coverage.
Output findings with severity labels. This review is ADVISORY - report only." \
--max-turns 10 \
--allowedTools Read,Grep,Glob \
--output-format json > /tmp/review.json
jq -r '.result' /tmp/review.json
COST=$(jq -r '.total_cost_usd' /tmp/review.json)
echo "Run cost: $COST"
awk -v c="$COST" 'BEGIN { if (c+0 > 0.50) print "##vso[task.logissue type=warning]Claude run
cost $" c }'
displayName: Claude advisory review
env:
ANTHROPIC_API_KEY: $(ANTHROPIC_API_KEY)
```

Step 3 - trigger it with a deliberately bad PR:

```
git checkout -b test-review
cat > src/bad-code.js <<'EOF'
const API_KEY = "sk-1234567890abcdef";
function runQuery(userInput) {
return `SELECT * FROM users WHERE name = '${userInput}'`;
}
module.exports = { runQuery };
EOF
git add -A && git commit -m "test: add review-bait file"
git push -u origin test-review
```

Open the pull request in the Azure DevOps web UI (or `az repos pr create --source-branch test-review --target-branch main --title "Test: Claude advisory review"` if you have the Azure CLI with the `azure-devops` extension).

Expected output marker: the pipeline runs on the PR and the review appears in the job log, flagging the hardcoded secret and the SQL injection with severities. **The PR is still completable** - advisory

informs, never blocks.

> ***To surface it as a PR comment rather than a log entry**, post the result to the `pullrequests/{id}/threads` REST endpoint using `$(System.AccessToken)`. That is plumbing, not learning - the review itself is already done by the time you get there.*

Step 4 - verify advisory, then clean up. Abandon the PR without completing it, and delete the branch.

GitHub Actions (alternate)

Same contract, less plumbing, because the official action handles the comment posting:

```
name: Claude PR Review
on:
pull_request:
types: [opened, synchronize]
jobs:
review:
runs-on: ubuntu-latest
permissions: { contents: read, pull-requests: write, issues: read }
steps:
- uses: actions/checkout@v4
with: { fetch-depth: 0 }
- uses: anthropics/claude-code-action@v1
with:
anthropic_api_key: ${{ secrets.ANTHROPIC_API_KEY }}
prompt: |
Review this pull request for security, quality, and test coverage.
Post findings as a PR comment with severity labels.
If a review comment from you already exists for this commit, do not post a duplicate.
This review is ADVISORY - do not fail the check for findings.
claude_args: "--max-turns 10 --allowedTools Read,Grep,Glob"
```

Notice what is identical across both: the `--max-turns` bound, the read-only `--allowedTools` allowlist, and the advisory instruction. Those three are the actual lesson. **What differs is only who posts the comment** - the GitHub action does it for you; on Azure DevOps you either read the log or make one REST call. Do not let the YAML dialect obscure that.

> ***The no-duplicate instruction matters on both.** `synchronize` on GitHub and the PR trigger on Azure both re-fire the job on every push to the branch - without it you accumulate a review comment per push.*
>
> ***Advisory vs gating:** start advisory; graduate to gating (exit-code enforcement like Phase 1's script + branch protection) only after the team calibrates thresholds. Too strict too early = developers route around the bot.*

FINISH MARKER - Success Checklist

- [ ] `claude -p` ran with BOTH `--allowedTools` and `--max-turns`
- [ ] JSON envelope parsed: `.result`, `.total_cost_usd`
- [ ] `scripts/ci-review.sh` runs and exits 0/1 on a score threshold
- [ ] Cost-bound fail-safe in place: turn-cap hit -> exit 2, job stops (no silent retry)
- [ ] Idempotency guard proven: second run on the same SHA printed SKIP, exit 0, zero cost
- [ ] You can state where a loop-to-invoice alert fires and who it pages

- [ ] `anthropics/claude-code-action@v1` workflow committed
- [ ] Advisory PR comment received on the bad-code PR (or watched in demo)
- [ ] You can state when to move advisory -> gating

If Stuck

Problem	Recovery
Headless can't read files	You omitted `--allowedTools`
`jq` errors on review.json	The response isn't bare JSON - extract `.result` first with `jq -r '.result'`
Script always prints SKIP	Stale marker: `rm -rf .ci-review/` or commit a change so HEAD moves
`.subtype` is empty on your build	Envelope fields vary by version - the `num_turns` check still catches the bound; verify field names with `cat review.json`
Action fails at auth	`gh secret list` must show ANTHROPIC_API_KEY; re-set it
No PR comment appears	Check `permissions: pull-requests: write` in the workflow
`bc` not installed	Ask Claude to rewrite the threshold check as an integer compare
Runs slow/expensive	Lower `--max-turns`; scope the prompt to changed files only

Debrief Questions

1. What do `--max-turns` and `--allowedTools` each protect you from in CI?
2. Why must hitting the turn cap exit **non-zero** - what does a "successful" truncated run invite your CI to do?
3. Why start advisory instead of gating on day one?
4. Which credential and billing path does this CI run use, and where does the loop-to-invoice alert fire before the platform team sees the bill?